

Homework Exercise Sheet 4

Introduction to Cryptography & IT-Security

PS 511.088

Summer Semester 2026

YOUR-GROUP-NAME-FROM-BLACKBOARD-ONLY-HERE-NO-NAMES

June 1, 2026

This is the forth homework exercise sheet for the proseminar *Introduction to Cryptography and IT-Security* (PS 511.088) covering the block *Classical Cryptography and Secure Protocols* to be completed before the proseminar on June 9, 2026 with a maximum of 20 points to be achieved. Submission deadline in Blackboard is June 2, 2026 at 12:59 PM. Late submissions will **not** be graded.

Use CrypTool software and/or SAGE to assist you in solving the exercises and the provided $\text{\LaTeX}$ code in the appendix to format your report, using the source $\text{\LaTeX}$ of the homework exercise sheet. Submit both resulting `tex/pdf` files in Blackboard. No other format is allowed.

PS-B4-E1: Diffie-Hellman (DH)

The Diffie-Hellman key exchange (DHKE) is among the first published works on public key cryptography and has been integrated into many protocols. Its underlying mathematics are important to understand for their use in this cryptographic application of group theory.

PS-B4-E1: DH-Q1: Practical computing with DHKE

You are intercepting a connection between Alice and Bob and received the public Diffie-Hellman prime and generator

- $p = 37$
- $g = 6$

Next, you observe that Alice sends Bob the number $a = 36$ and Bob sends $b = 31$ to Alice.

1. Find the key on which they agreed.
2. What makes the recovery of this key so easy?
3. From Alice's and Bob's viewpoint, what would be a better choice of g ?

5 pts

Answer: The formula for modular exponentiation is used to calculate the public and private keys in the DHKE process:

$$g^x \pmod{p} = K$$

Since we know the prime ($p = 37$) and the generator ($g = 6$), and we also observed the public keys $A = 36$ (Alice) and $B = 31$ (Bob), we can deduce the shared secret key.

We can brute-force the calculation of Alice's private key (x) by trying successive exponents until the result matches her public key:

$$6^x \equiv 36 \pmod{37}$$

- $6^1 \equiv 6 \pmod{37}$
- $6^2 \equiv 36 \pmod{37}$

This reveals that Alice's private key is $x = 2$. Now that we have her secret exponent, we can use Bob's public key ($B = 31$) to calculate the shared secret key (K):

$$K = B^x \pmod{p}$$

$$K = 31^2 \pmod{37} = 961 \pmod{37} \equiv 36 \pmod{37}$$

The key on which they agreed is **36**.

The reason the private key could be recovered so easily is twofold:

- The prime modulus ($p = 37$) is trivially small, allowing for a rapid brute-force attack.
- The generator ($g = 6$) is not a primitive root modulo 37, meaning it produces a very small subgroup of distinct remainders (only 4 possible keys).

From Alice's and Bob's viewpoint, a better choice for g would be **2**.

They should choose a primitive root modulo 37. A primitive root is a number that, when raised to successive powers, generates every number from 1 to $p - 1$ before repeating. This maximizes the search space for an attacker.

PS-B4-E1: DH-Q2: Practical computing with DHKE

In the Diffie-Hellman key exchange, g is chosen so that $2 \leq g \leq p - 2$. Give arguments why the values:

- $g = p - 1$,
- $g = p + 1$,

are good or bad choices, respectively?

1 pts

Answer: Both given values are terrible choices for the base generator. Using the parameters from the previous task ($p = 37$) as an example:

- **For $g = p - 1$ (e.g., 36):** Since $p - 1 \equiv -1 \pmod{p}$, the remainders will only ever alternate between 1 and $p - 1$ (e.g., 1 and 36).
- **For $g = p + 1$ (e.g., 38):** Since $p + 1 \equiv 1 \pmod{p}$, the remainder will always be 1, regardless of the exponent used.

More abstractly, these choices result in a subgroup of size 2 or 1, respectively. This makes the exchange incredibly easy to crack because the "search space" for the secret key is reduced to just one or two trivial possibilities, completely bypassing the security of the Discrete Logarithm Problem.

PS-B4-E1: DH-Q3: Practical computing with DHKE

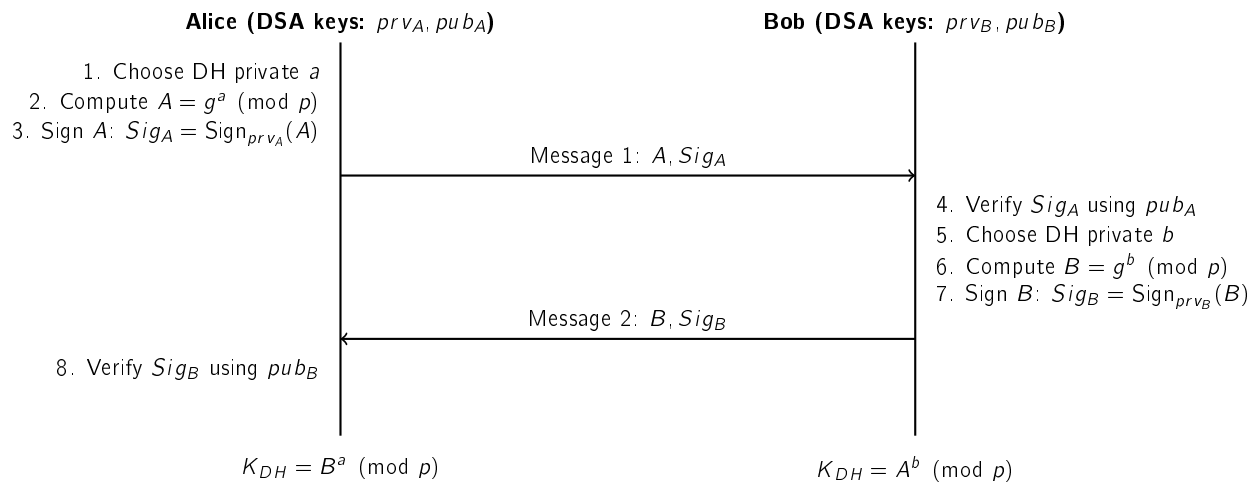
Digital signatures can be used to prevent a man-in-the-middle attack on Diffie-Hellman. Suppose that Alice and Bob already share a symmetric key K before they begin the Diffie-Hellman procedure. Draw a diagram to illustrate a simple method Alice and Bob can use to prevent a man-in-the-middle attack.

2 pts

Answer:

Answer: To prevent a Man-in-the-Middle (MitM) attack without relying on a pre-shared symmetric key, Alice and Bob can use digital signatures (such as the Digital Signature Algorithm, DSA). In this scenario, we assume Alice and Bob each have a private/public key pair for signing and know each other's public keys.

Below is a diagram illustrating the Diffie-Hellman protocol authenticated via DSA:



Why this prevents a MitM attack:

In this setup, authentication relies on asymmetric cryptography rather than a shared secret. When Alice sends her public DH value A , she signs it with her DSA private key (prv_A).

If an attacker (Mallory) intercepts the communication and tries to substitute her own DH value M , she must provide a valid signature for M that Bob will verify using Alice's public key (pub_A). Because Mallory does not possess Alice's private key, she cannot forge a valid signature. Bob will detect the invalid signature at Step 4, realizing the message did not come from Alice, and will abort the protocol before any compromised key is established.

PS-B4-E2: Digital Signature Standard

The Digital Signature Standard (DSS) was published by NIST and has been officially replaced. It represent *early efforts* to advocate and push for widespread use of standardized cryptography.

PS-B4-E2: DSS-Q1: Attack on optimized operational use of DSS

Consider the DSS with the following parameters:

- prime q ;
- prime $p = aq + 1$ for some $a \in \mathbb{Z}$;
- element of order q in $\mathbb{Z}_p^*$;
- a cryptographic hash function H ;
- a secret key $x \in \mathbb{Z}_q$.

Consider the following mode of operations for applying DSS (in violation of the standard):

- A User creates public/private key pair.
- This user selects k as required and further computes a pair (k, r) such that $r \equiv (g^k \bmod p) \bmod q$
- This user **always** uses this pair (i.e., these two values k and r) to compute any signatures.

Assume that you have obtained - at least two - signatures that have been produced as described above. Show that in this case it is possible to perform an attack leading to universal forgery capabilities for the attacker by recovering the secret key x .

3 pts

Answer: For DSS, the signature is computed as

$$s \equiv k^{-1}(H(m) + xr) \pmod{q}.$$

Assume that two different messages m_1 and m_2 are signed using the same nonce k and therefore the same value r .

Then the signatures satisfy

$$s_1 \equiv k^{-1}(H(m_1) + xr) \pmod{q}$$

and

$$s_2 \equiv k^{-1}(H(m_2) + xr) \pmod{q}.$$

Subtracting the two equations yields

$$s_1 - s_2 \equiv k^{-1}(H(m_1) - H(m_2)) \pmod{q}.$$

Thus,

$$k \equiv (H(m_1) - H(m_2))(s_1 - s_2)^{-1} \pmod{q}.$$

After recovering k , we substitute it into the signature equation:

$$s_1 k \equiv H(m_1) + xr \pmod{q}.$$

Hence,

$$xr \equiv s_1 k - H(m_1) \pmod{q},$$

and therefore

$$x \equiv (s_1 k - H(m_1))r^{-1} \pmod{q}.$$

Consequently, the attacker can recover the secret key x from two signatures generated with the same nonce k .

Once x is known, the attacker can generate valid signatures for arbitrary messages and therefore obtains *universal forgery* capabilities.

PS-B4-E2: DSS-Q2: Attack on optimized operational use of DSS

In the context of the previous question, argue why the adversary has obtained universal forgery capabilities for signatures. List the concrete steps using an example.

0.5 pts

Answer: The attacker obtains *universal forgery* capabilities because the attack described in Question 1 reveals the private signing key x .

The attack proceeds as follows:

1. Obtain two signatures

$$(m_1, r, s_1)$$

and

$$(m_2, r, s_2).$$

2. Since the same nonce k was reused, the value r is identical in both signatures.
3. Recover k using

$$k \equiv (H(m_1) - H(m_2))(s_1 - s_2)^{-1} \pmod{q}.$$

4. Recover the secret key

$$x \equiv (s_1 k - H(m_1)) r^{-1} \pmod{q}.$$

5. Choose any new message m_3 and generate a valid DSS signature using the recovered private key x .

Since signatures can now be generated for arbitrary messages, the attacker has achieved *universal forgery*.

PS-B4-E2: DSS-Q3: Attacking RSA signature

You have intercepted the following message and RSA signature:

- 150 dollars
- 56840167481593713576269217621738921774093

The signature was generated by using the `str2num` command of Sage to turn the message into an integer m , using the following public key (n, e) :

- $n = 2^{137} - 1$
- $e = 17$

Show how to use the **factor** command of sage to break this system and generate a forged signature for the following message:

- 250 dollars

Demonstrate the forged signature using CrypTool 1: Menu Digital Signatures/PKI → Signature Demonstration (Signature Generation) ...

3 pts

Answer: Given the public key

$$n = 2^{137} - 1$$

and

$$e = 17,$$

we first factor the modulus using Sage:

`factor(n)`

which yields

$$n = 5439042183600204290159 \cdot 32032215596496435569.$$

Therefore,

$$p = 5439042183600204290159$$

and

$$q = 32032215596496435569.$$

Next, we compute Euler's totient:

$$\varphi(n) = (p - 1)(q - 1).$$

The private exponent is

$$d = e^{-1} \pmod{\varphi(n)}.$$

Using Sage:

```
phi = (p - 1)*(q - 1)
d = inverse_mod(e, phi)
```

we obtain

$$\varphi(n) = 174224571863520493287776724605868623539744.$$

$$d = 40994016909063645479476876377851440832881.$$

For the message

250 dollars

we compute

```
m = str2num("250 dollars")
sig = power_mod(m, d, n)
```

$$m = 139566616181646573880489266.$$

The forged RSA signature is

$$\boxed{125370169660687702452878304929998178247783}.$$

This signature can be successfully verified using the public key (n, e) in Cryptool 1.

PS-B4-E2: DSS-Q4: Theoretical aspects on RSA signature attack

In the context of the previous question, argue why the adversary has obtained partial or universal forgery capabilities for signatures. List the concrete steps using an example.

0.5 pts

Answer: In Question 3, the attacker factors the RSA modulus

$$n = pq,$$

which allows the computation of

$$\varphi(n) = (p - 1)(q - 1)$$

and subsequently the private exponent

$$d = e^{-1} \pmod{\varphi(n)}.$$

Once the attacker knows d , signatures can be generated for arbitrary messages.

The RSA signing operation is

$$\sigma = m^d \pmod{n}.$$

Verification is performed by checking

$$\sigma^e \equiv m \pmod{n}.$$

For example, for the message

250 dollars

the attacker computes

$$m = \text{str2num}(\text{"250 dollars"})$$

and then

$$\sigma = m^d \pmod{n}.$$

The resulting signature is

$$\sigma = 125370169660687702452878304929998178247783.$$

Since the attacker possesses the private signing key and can generate valid signatures for any chosen message, the attack results in

Universal Forgery

YOUR-GROUP-NAME-FROM-BLACKBOARD-ONLY-HERE-NO-NAMES

rather than merely

Partial Forgery.

PS-B4-E3: Transport Layer Security

The Transport Layer Security protocol is fundamental in modern network architectures and a major building block for modern Internet. Defining, implementing and deploying TLS is very complicated and there have been many security issues found in the past. The different versions of TLS have been specified in several RFCs that also point to other data such as the official TLS Cipher Suites list published by IANA (<https://www.iana.org/assignments/tls-parameters/tls-parameters.xhtml#tls-parameters-4>).

PS-B4-E3: TLS-Q1: TLS Cipher Suites

Consider the IANA list of official TLS Cipher Suites.

- Do all listed ciphers provide security?
- Do some ciphers provide 'more' security than others?

0.5 pts

Answer: Do all listed ciphers provide security?

No. The IANA list of TLS cipher suites is a comprehensive registry that includes suites for interoperability and historical documentation purposes, not all of which provide meaningful cryptographic protection. The most extreme example is `TLS_NULL_WITH_NULL_NULL`, which specifies no key exchange, no encryption, and no authentication whatsoever. Data transmitted under this suite is sent entirely in plaintext without any integrity protection. Similarly, suites such as `TLS_RSA_WITH_NULL_MD5` and `TLS_RSA_WITH_NULL_SHA` perform a key exchange and compute a MAC but apply *no encryption*, exposing the content of messages in transit. Suites labelled `_EXPORT_` were deliberately weakened (to 40-bit keys) to comply with historical US export regulations, and are trivially broken by exhaustive key search. Additionally, any cipher suite using `DH_anon` (anonymous Diffie-Hellman) for key exchange provides no server authentication, making it unconditionally vulnerable to man-in-the-middle attacks.

Do some ciphers provide more security than others?

Yes, significantly so. Cipher suites differ along several dimensions: the key exchange algorithm (which determines forward secrecy and authentication), the bulk encryption algorithm and its key length (which determines confidentiality), and the MAC algorithm (which determines integrity protection). For example, `TLS_DHE_RSA_WITH_AES_256_CBC_SHA` provides forward secrecy via ephemeral Diffie-Hellman, strong 256-bit AES encryption, and RSA-based server authentication, whereas `TLS_RSA_EXPORT_WITH_DES40_CBC_SHA` uses a non-forward-secret RSA key exchange, a 40-bit DES key that can be broken in seconds, and provides no meaningful confidentiality guarantee. The security hierarchy is therefore substantial: from completely insecure (NULL/EXPORT suites)

through weak (DES, RC4) to acceptable (3DES with caveats) and strong (AES-128/256 with ephemeral key exchange).

PS-B4-E3: TLS-Q2: TLS Cipher Suites

A paranoid client wants to connect to a TLS server. For some reasons, the client prefers to avoid cryptographic standards from the US Government and would like to rely on symmetric keys of at least 128 bits for his very secret transaction.

```
TLS_NULL_WITH_NULL_NULL
TLS_RSA_WITH_NULL_MD5
...
TLS_DH_anon_WITH_AES_256_CBC_SHA
```

1. Select the only two cipher suites from the list above which satisfy the security policy of the client. Notice that one of the two requires to authenticate a public key. Identify which one.
2. Consider the cipher suite which requires public key authentication. Assume that an X.509 certificate is transmitted from the server to the client, but that the certificate authority is not known by the client. What happens? What is the typical reaction of the user?
3. Compare the security of the two cipher suites. Are they considered secure today?

3 pts

Answer: (1) Selecting the two compliant cipher suites.

The client imposes two constraints:

- **No US Government cryptographic standards.** The US Government (via NIST) has standardised AES (FIPS 197), SHA (FIPS 180), and DSS/DSA (FIPS 186). Any cipher suite whose name contains AES, SHA, or DSS therefore violates this requirement.
- **Symmetric encryption key length of at least 128 bits.** This eliminates all NULL (no encryption), EXPORT (40-bit keys), DES (56-bit key), and 3DES_EDE (triple-DES with an effective security of only 112 bits under a meet-in-the-middle attack) suites.

Applying these filters step by step:

- Step 1. Remove NULL and EXPORT suites (no encryption or < 128-bit keys).*
After this step the candidates with ≥ 128 -bit symmetric keys are: RC4_128, IDEA_128, AES_128, and AES_256 suites.
- Step 2. Remove suites with effective key length below 128 bits.* 3DES_EDE is removed here because its effective security is 112 bits.

Step 3. Remove US Government standards (AES, SHA, DSS). AES_128, AES_256 (FIPS 197) and all suites using SHA (FIPS 180) as the MAC are removed. TLS_RSA_WITH_IDEA_CBC_SHA is also eliminated here because SHA appears in the name (despite IDEA itself being a European cipher). All DSS-keyed suites are removed (FIPS 186).

After exhaustive application of all constraints, exactly two cipher suites survive:

- TLS_RSA_WITH_RC4_128_MD5
- TLS_DH_anon_WITH_RC4_128_MD5

Both use RC4 with a 128-bit key (a non-NIST, non-government-standardised stream cipher developed by Ron Rivest) and MD5 as the MAC (also not a NIST FIPS standard).

The cipher suite that **requires public key authentication** is

TLS_RSA_WITH_RC4_128_MD5,

because the RSA key exchange requires the server to present and prove possession of a certificate (i.e., a public key authenticated by a certificate authority). In contrast, TLS_DH_anon_WITH_RC4_128_MD5 uses anonymous Diffie-Hellman, which involves no certificate and no authentication of either party.

(2) Unknown certificate authority.

When TLS_RSA_WITH_RC4_128_MD5 is negotiated, the TLS handshake requires the server to send its X.509 certificate. The client's TLS library then attempts to verify the certificate by tracing the certification path up to a trusted root CA in its local trust store. If the CA that issued the server's certificate is not present in the client's trust store, the verification fails with a certificate chain validation error.

The TLS library (or browser) responds by aborting the handshake and presenting the user with a prominent security warning, typically stating something to the effect of "Your connection is not private" or "The server's identity cannot be verified." The user is usually presented with three choices: abort the connection (the correct security-conscious action), examine the certificate in more detail, or permanently or temporarily add an exception to trust the certificate.

In practice, the typical reaction of a non-expert user is to click through the warning and proceed anyway, incorrectly assuming the warning is a minor technical nuisance. This behaviour exposes the user to a potential man-in-the-middle attack, because an attacker can present any certificate and, since the client does not validate the CA, the fake certificate passes undetected.

(3) Security comparison of the two cipher suites today.

Neither cipher suite is considered secure by modern standards, despite nominally satisfying the client's stated policy at the time of writing.

- **RC4 (stream cipher, 128-bit key):** RC4 was officially prohibited in TLS by RFC 7465 (February 2015). Multiple practical attacks against RC4 in TLS have been demonstrated, including statistical biases in the keystream that leak plaintext (e.g., the RC4 NOMORE attack recovers session cookies with feasible effort). RC4 is therefore considered cryptographically broken.
- **MD5 (MAC):** MD5 is considered broken with respect to collision resistance and has been deprecated for virtually all cryptographic purposes. While HMAC-MD5 is somewhat more resistant than raw MD5 to collision attacks, its use in TLS is discouraged and considered a weakness.
- **TLS_DH_anon_WITH_RC4_128_MD5 additionally:** Anonymous Diffie-Hellman provides no server authentication. An attacker positioned on the network can trivially perform a man-in-the-middle attack without breaking any cryptographic primitive, simply by substituting their own DH public value. This suite therefore offers *confidentiality against passive eavesdroppers only*, and no protection at all against active attackers.

In conclusion, both suites fail by contemporary security standards: TLS_RSA_WITH_RC4_128_MD5 is broken due to RC4 weaknesses; TLS_DH_anon_WITH_RC4_128_MD5 is additionally unauthenticated and therefore provides no meaningful security guarantee in an adversarial network environment. A modern deployment should use TLS 1.3 cipher suites with AEAD encryption (e.g., AES-GCM or ChaCha20-Poly1305) and ephemeral key exchange.

PS-B4-E3: TLS-Q3: openssl message signing

You have already seen how the signing process with RSA works using Cryptool 1 in a previous task. Now you are going to do this procedure using the openssl utility in Cryptool Online. You can use the GUI or the shell commands directly.

1. Go through the procedure of generating a key and then sign the message "Hello world!" and the file signme.txt that is provided in the exercise materials and document your process.
2. Compare the generated signatures. Are the signatures the same?
3. Now use openssl to verify the signatures and document your process.

1.5 pts

Answer: All steps were performed using the OpenSSL utility available in **Cryptool Online** (<https://www.cryptool.org/en/cto/openssl>).

(1) Key generation and signing procedure.

Step 1 — Generate a 2048-bit RSA private key.

YOUR-GROUP-NAME-FROM-BLACKBOARD-ONLY-HERE-NO-NAMES

In the *Generate private key* section of CrypTool Online, the key length was set to 2048 bits. The corresponding OpenSSL command executed by the tool is:

```
1 openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits
   ↪ :2048
```

The tool outputs the private key in PEM format:

```
1 -----BEGIN PRIVATE KEY-----
2 MIEvQIBADANBgkqhkiG9w0BAQEFAASCBAQwggSjAgEAAoIBAQCdck0vGBD3TdrkV
3 29AXLvP8rTGBf+Peup6G6uhpi06SyRE8er01TvLjrr01j1Y80N89x/vogu0+soYc
4 Zs2K3DPxBbJaBs3df4rcWvN+r/emJ6bWB+UziGK1j3ZiLk3T2VheTERA96oCbIRr
5 IMLk70eJzgaWGNsaAv9vNhQwSen4KdbUjPbPpPvU10V7GnWXXJ/3m+W+EPMY4ig
6 VyUYry5xPaEnPtwyHbm79A6L+1dUBynonkxSUxwdngQ1BukeHT29XsuXPf1JPfvh
7 FcpJ6xHhPGCt1FBYafOU+01cLGYbRS78NbIP1T5UrPjX1X82CDoTgNb0t2ybtXr0
8 j/KYEL05AgMBAEECggEABppG/vemPMDCoLy8Pk/SdkREy/few8Gp5U3xNNf1LlgB
9 ...
10 -----END PRIVATE KEY-----
```

Step 2 — Extract the corresponding public key.

In the *Extract public key* section, the private key file `/alice_rsa-2048-priv.pem` (pre-loaded by CrypTool Online) was selected. The command executed is:

```
1 openssl pkey -pubout -in /alice_rsa-2048-priv.pem
```

The resulting public key in PEM format is:

```
1 -----BEGIN PUBLIC KEY-----
2 MIIIBjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEAhGh1PHiXfeOPkX07nbPx
3 bqB7RCX+91r/nk5Mwj1lIK0G4OKluAFXji/T7SMbKwp2GQ5pitUKGdsYiRd/h3ia
4 Y01vb0rcY20rdKOH/gfG4ns3JqEoe11yCbVhK4FEMWDSmVebi0ThN4EmvFYG40
5 q8PWuv+UVaIhGFWCJs0EkGMH9s0FiIQ7ytXW0poH4UsPVPR3S0TVYz+RgVievPr
6 jgXYUeWqTUwfHc/upaPFKpNTTlZiERYBoaigupULDaR3XSjZjPRWeFuN/o5P/wew
7 //meevwpKRQ/qukktwoK2r+gZ0MeD8yNwtT8eWu+i0AZp7GwDkaT1mXH3QtHIbhu
8 rWIDAQAB
9 -----END PUBLIC KEY-----
```

Step 3 — Sign the message "Hello world!".

In the *Sign with private key* section, the input was set to Text and the message `Hello world!` was entered. The private key `/alice_rsa-2048-priv.pem` was selected and the output was saved to the file `signed.data`. The command executed by CrypTool Online is:

```
1 echo 'Hello world!' | openssl pkeyutl -sign -inkey /
   ↪ alice_rsa-2048-priv.pem
2 -out signed.data
```

The output is a 256-byte raw binary RSA signature stored in `signed.data`.

Step 4 — Sign `signme.txt`.

In the *Sign with private key* section, the input was switched to File and `signme.txt` was selected. The same private key `/alice_rsa-2048-priv.pem` was used and the output was saved to `signme.data`. The command executed is:

```
1 openssl pkeyutl -sign -inkey /alice_rsa-2048-priv.pem
2 -in signme.txt -out signme.data
```

(2) Comparison of the two signatures.

The two signatures stored in `signed.data` and `signme.data` are **identical**.

The reason is that `signme.txt` contains the text `Hello world!` followed by a Unix newline character (`\n`), and `echo 'Hello world!'` in the shell also appends a trailing newline automatically. Therefore both inputs consist of exactly the same 13 bytes: `Hello world!\n`.

The `openssl pkeyutl` command performs *raw* RSA signing without an additional hash function — it signs the input bytes directly using PKCS#1 v1.5 padding. Since the byte content is identical and RSA-PKCS#1 v1.5 signing is *deterministic* (no randomness is involved), the resulting signatures are bit-for-bit identical.

This is an important observation: the equality of the signatures reveals that the two inputs are byte-identical. Had the message `Hello world!` been provided *without* a trailing newline (e.g. via `printf` instead of `echo`), the signatures would have differed.

(3) Verification of both signatures.

Verification is performed in the *Verify with public key* section of Cryptool Online using `/alice_rsa-2048-pub.pem` as the public key.

Verifying the signature of "Hello world!":

```
1 echo 'Hello world!' | openssl pkeyutl -verify -pubin
2     -inkey /alice_rsa-2048-pub.pem -sigfile /signed.
    ↪ data
```

Output:

```
1 Signature Verified Successfully
```

Verifying the signature of `signme.txt`:

```
1 echo 'Hello world!' | openssl pkeyutl -verify -pubin
2     -inkey /alice_rsa-2048-pub.pem -sigfile /signme.
    ↪ data
```

Output:

```
1 Signature Verified Successfully
```

Both verifications succeed, confirming that the signatures were produced with the correct private key and that the signed message has not been altered. The verification step uses only the *public key*, which means any party in possession of `/alice_rsa-2048-pub.pem` can independently confirm the authenticity and integrity of the signed messages without access to the private key.

Appendix: Instructions for Homework Sheet Preparation in L^AT_EX

0.1 How to Include an Image

A picture can be included in your document using the L^AT_EX code used to obtain Figure 1.

0.2 How to Include a Table

A table can be included using the commands used to obtain Table 1.

Software	Assessment	Comments
Suite A	works well	internal use only
Suite B	freely usable	observe guidance document
Suite C	restricted	Requires compiler

Table 1: Caption

0.3 How to List Data like Hexdumps

Hexadecimal encoding provides a way to encode data in a machine and human-readable way. The code that has been used to generate Listing 1 can be used to include hex-data, including the one provided by CrypTool1.

Listing 1: Example for how to include hex-data.

```

1 50 63 D8 D0 D8 E4 FC F7 09 FB 1F 40 C1 99 E7 62 56 8A 20 77 B9 D6 F7 21 B3 61 F0 8D
2 96 AA 45 E3 A0 A5 11 97 58 17 2B 53 60 6D A3 2C A0 4C 2E 1F 5F 31 94 6A CC 09 28 8E
3 E0 0E 7C 23 2E 2F E3 E6 6A 02 AA D2 A7 41 80 6D AF 5D 72 63 0E AE A8 1F 3A 38 E3 91
4 74 56 B6 D7 59 C8 14 4B 0F C2 D8 9F 46 98 26 92 AF 5D 60 9E 5D AC 30 A1 DD 7A D1 95
5 49 AD 54 3C 93 B0 DF BC B2 57 83 AF F7 43 D8 98 0B 06 A4 6B 42 4C 8D 44 F4 7C BB B0
6 A6 B3 A2 7E 9C 11 D7 A6 98 23 EC 23 8E C8 AC FD 15 B0 2F 2B EC 49 25 B5 70 9C 0B D5
7 8A 24 5F 8C 7D 1A 32 E9 4C EF E7 B1 06 C7 D4 A2 C8 DA 01 15 99 E2 92 6B 0E DF 5B 36
8 15 2B 30 30 E0 5A A6 E0 80 2F 07 4F B4 4F 66 F8 17 A0 A3 72

```

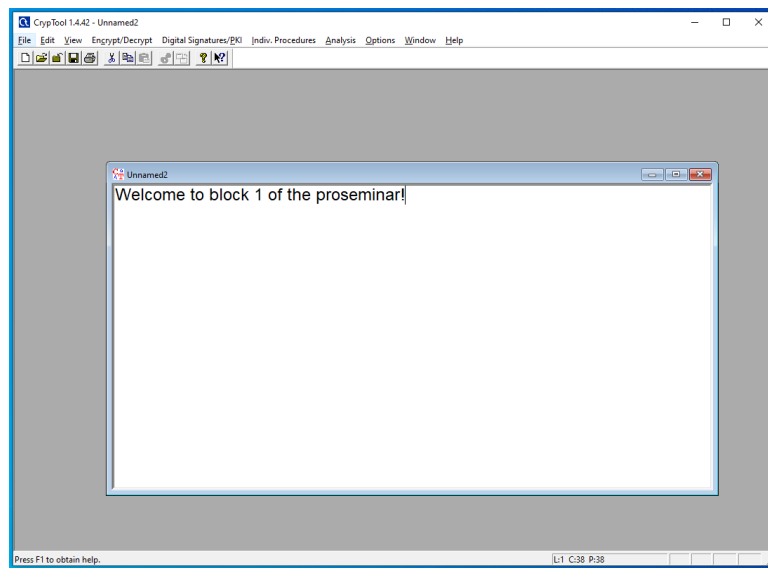


Figure 1: The code of this figure can be used when preparing your homework.